# NHDW DEVELOPMENT METHODOLOGY

**PREPARED FOR**

Ethiopian Ministry of Health

**PREPARED BY**

Yonas Tesfaye
Eyasu Ashebier
Abenezer Tesfaye

Data.FI Ethiopia

## Executive Summary

The health sector has entered a phase where data volume, system diversity, and analytical ambition have significantly increased the need to improve the capacity of existing data platforms. Over the past decade, the Ministry of Health and partners have invested heavily in digital health systems across service delivery, logistics, human resources, and community health. These investments have produced valuable data assets, but they now exist in fragmented silos, limiting their ability to collectively inform policy, planning, and patient care at national scale.

The National Health Data Warehouse (NHDW) is intended to address this structural gap. Its purpose is not simply to consolidate data, but to establish a secure, auditable, and scalable national platform that transforms disparate health datasets into a coherent, trusted, and actionable foundation for decision making. When fully implemented, the NHDW will serve as Ethiopia's single analytical backbone for health, supporting routine monitoring, advanced analytics, and future AI driven use cases while complying with national data governance and privacy regulations.

This document presents a phased, standards aligned technical approach for delivering the NHDW in line with the Ministry of Health's High Level Requirements, national eHealth Architecture, and data governance frameworks.

## Background and Context

Ethiopia's health information ecosystem spans more than eighty systems across clinical care, public health programs, supply chain, registries, and external sectors. These systems were designed to solve specific operational problems and have largely succeeded in doing so within their domains. However, they were not designed to function as a unified analytical environment.

As a result, critical questions that require cross system analysis such as linking commodity availability to service delivery, tracking longitudinal patient outcomes, or integrating health data with demographic and environmental signals require manual, ad hoc, and often non reproducible processes. This limits timeliness, consistency, and confidence in insights used for policy and program decisions.

At the same time, demand for data has evolved. Decision makers increasingly require near real time visibility, longitudinal analysis, and predictive insights rather than static reports. These expectations cannot be met sustainably without a foundational data platform that is architected for scale, auditability, and continuous evolution.

## Project History and Strategic Drivers

The Ministry of Health has articulated the need for a centralized health data platform through multiple strategic initiatives, including the national eHealth Architecture and alignment with broader eGovernment data programs. The NHDW High Level Requirements formalize this direction, defining a phased roadmap that prioritizes raw data centralization, interoperability, governance, and advanced analytics.

Key strategic drivers behind the NHDW include the need to establish a single source of truth for health analytics, reduce duplication of data integration efforts across programs, improve data quality and traceability, and enable advanced use cases such as predictive modeling, early warning systems, and integrated performance monitoring across the health sector.

Rather than replacing existing systems, the NHDW is positioned as an enabling layer that respects current investments while unlocking their combined value through standardized ingestion, modeling, and governance.

## Requirements Overview

The High Level Requirements define **data quality, functional**, **non-functional & infrastructure** expectations for the NHDW. Functionally, the platform must support ingestion from diverse source systems, preserve raw data fidelity, enable longitudinal and cross domain analysis, and provide secure access for a wide range of users. Non functionally, it must meet stringent requirements for security, privacy, auditability, scalability, reliability, and cost effectiveness under Ethiopian law and national data governance policies.

Critically, the requirements mandate a hybrid architecture and Data Vault 2.0 methodology to ensure flexibility, historical traceability, and resilience to change. This reflects an understanding that health data systems, policies, and analytical needs will continue to evolve over time.

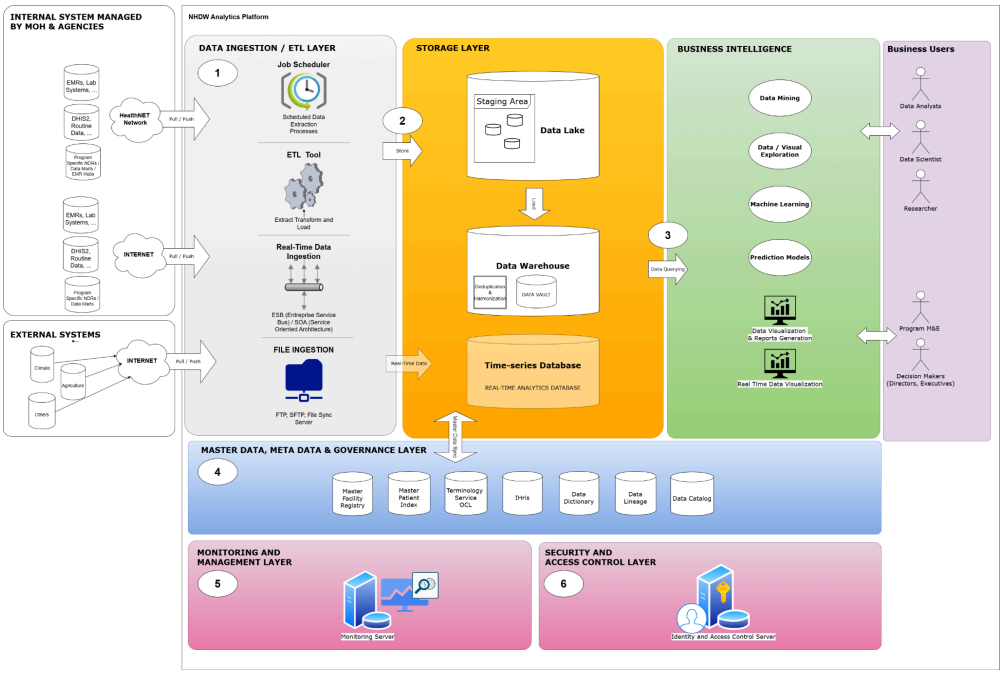
## Intro to the NHDW

The National Health Data Warehouse (NHDW) is to be designed as an integrated analytics platform that consolidates health and health related data from across Ethiopia into a single, governed environment. Rather than operating as a standalone system, the NHDW functions as a national analytical backbone, enabling data from diverse operational systems to be ingested, structured, and made available for reporting, advanced analytics, and decision support.

The architecture reflects the Ministry of Health's High Level Requirements for Phase One, emphasizing secure data ingestion, preservation of raw data, scalable storage, and controlled access for multiple analytical use cases. It is aligned with Ethiopia's national eHealth Architecture and is designed to interoperate with broader eGovernment initiatives, including the National Data Warehouse, without duplicating existing system responsibilities.

As illustrated in the architecture, the NHDW is organized into clearly defined layers. Source systems managed by the MoH, agencies, and external partners feed into standardized ingestion pipelines. Data is stored and processed across a layered storage environment, progressing from raw staging and data lake storage into a governed data warehouse. Business intelligence, analytics, and AI capabilities are delivered through a dedicated consumption layer, while master data management, metadata, monitoring, and security are applied as cross cutting capabilities to ensure data quality, traceability, compliance, and operational reliability across the platform.

**The architecture:**



## Proposed Approach

In response to these drivers and requirements, this document proposes a layered National Health Data Warehouse architecture that progresses from raw data centralization to governed analytics and AI enablement. The solution is intentionally modular, phased, and standards based.

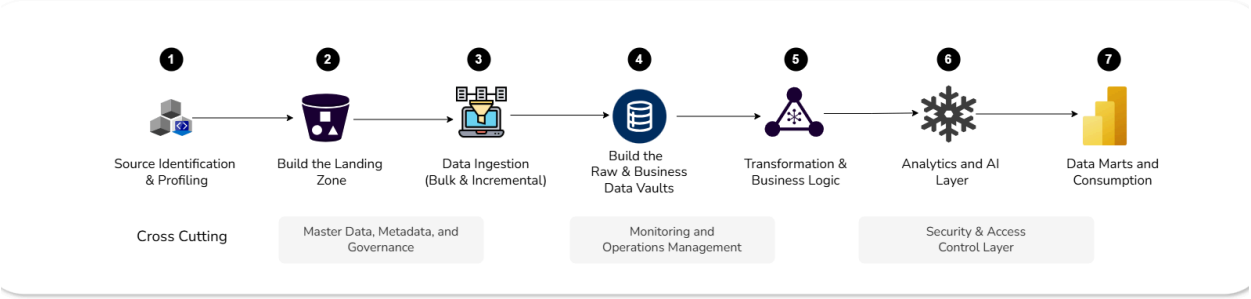
It begins with structured source system identification and profiling, followed by a secure data lake for raw data preservation, a Data Vault based warehouse for historical and auditable integration, and governed data marts for analytics and decision support. Cross cutting layers for master data management, metadata, security, monitoring, and governance ensure trust, compliance, and operational sustainability.

The sections that follow detail this solution in depth, starting with Data Source Identification and Profiling, and progressing through ingestion, modeling, governance, analytics, and security. Each design decision is explicitly grounded in the Ministry of Health's requirements and the operational realities of Ethiopia's health information landscape.

The development of this data warehouse will reflect the Ethiopian MoH's development and implementation roadmap that specifies five development phases in the overall development:

1. Planning and requirement gathering
2. Data warehouse design and development
3. Data modelling and analysis
4. Deployment and maintenance, and
5. Quality assurance and implementation

**Methodology:**




The following sections describe the NHDW delivery methodology end to end, progressing from source system assessment through ingestion, modeling, governance, analytics, and operations. A phased data delivery flow from source to insight, with governance, security, master data, and operations applied consistently across every phase:

## Data Source Identification & Profiling

The data warehouse is going to integrate from various source systems to sync and load from each. The data source systems can be categorized based on the type of service they provide. While data sources can be grouped by functional area (e.g. Disease Management, Regulatory), for architectural and ingestion planning, we adopt the standardized eHealth Architecture (eHA) categories defined by the MoH.

When mapping out data sources, the goal is not just to list them, but to understand what role each plays in the bigger picture. The approach to structuring the source layer will allow us to create a clear, organized inventory of all potential data sources to inform subsequent design and integration phases.

In order to achieve this, it is essential to divide them into simple step by step methodologies that we can use to get a full understanding. We start by using three steps in which we categorize the data source systems based on their charactersics.

The methodology consists of:

1. **Categorising Sources by eHA Function**: Classify each source system into one of five standardized categories; Point of Service, Institution-Based Services, Shared Services, Population-Based HIS, and External Systems as defined by Ethiopia's national eHealth Architecture. This immediately clarifies the system's role, data type, and technical requirements from a design perspective.
2. **Profile with Standardized Attributes**: Document key technical and operational characteristics (e.g., data volume, update frequency, sensitivity, owner, access method) using a consistent template. This creates a comparable source system profile for every system. This will prevent a one size fits all technical decision.
3. **Prioritize by Strategic Value**: Rank systems for integration based on MoH defined criteria: public health impact, data maturity, coverage, and alignment with early MVP use cases (e.g., Malaria Control, Maternal Health). This ensures resources are focused where they deliver the highest value, fastest.

### Categorize by eHA Function

We will group the data source systems using the established Ethiopian eHealth Architecture categories aligning with the requirements document (Section 3.1 and 6.3.1). This provides a standardized framework that is again aligned with the requirement document for the project.

The eHA categories are the official, government endorsed blueprint for how digital health systems should be structured and integrated according to Ethiopia's national enterprise architecture.

**Point of Service Systems**: (e.g., eCHIS, SmartCare).
These are usually real-time, transactional systems capturing data at the frontline of care.

**Institution-Based Services**: (e.g., DHIS2, iHRIS).
These manage internal operations like reporting, HR, and logistics for specific institutions or the MoH.

**Shared Services**: (e.g., Master Facility Registry, National ID/Fayda).
These provide common, reusable services (like registries) across multiple systems.

**Population-Based HIS**: (e.g., DHS, HDSS).
These data sources provide aggregated or survey-based data for understanding population health trends.

**External Systems**: (e.g., Ethiopian Statistical Service, Meteorology Institute).
These are cross-sectoral data sources crucial for holistic analysis.

### Profile with Standardized Attributes

This template is supposed to help us figure out the "How", “Who”, and "When". We should be able to learn what is practical, what’s risky, and what is needed overall.

For example, knowing how often a source system data updates tells us if daily loads are enough or if you need something closer to real-time. Understanding data sensitivity tells us where masking or extra security is a must. Even basics such as “who owns this system” save headaches later when permissions or issues come up.

The point being we would need to clearly point what format the data is going to be, in what frequency the source system is able to push the data, the volume (size) of the data we would be expecting, and etc.

1. **Source System Name -** e.g. Dagu
2. **Description -** What business data does it hold and its primary purpose?
3. **Source** - Internal or External - in accordance with eHA
4. **Type** - Individual level or Aggregate level
5. **Owner** - Specific application and managing agency
6. **Access Method** - How do we connect? e.g., API, SFTP
7. **Data Structure** - Schema or Format e.g., Relational, FHIR, Flat File
8. **Data Update Frequency** - e.g., Real-time, Daily, Monthly
9. **Data Volume** - Size and estimated growth
10. **Data Sensitivity** - Contains PII/PHI? Classification
11. **Historical Data Load Plan** - Is one-time historical data required, and how many years?
12. **Incremental Update** - How do we get updates after the initial load? This would mean using API Endpoints, CDC logs, or Partition Loads
13. **Reliability -** Stability rating: High/Medium/Low
14. **Constraints** - Access/query limits, blackout periods
15. **Source Data Retention Policy** - Most likely kept in full as per the high level requirement doc.

### Prioritize by Strategic Value

Not all data sources need to be integrated simultaneously or all at once. Using the criteria outlined in the requirements (Section 2.8.1.2), we will work with the MoH Technical Working Group (TWG) to prioritize sources for initial phases.

Key prioritization factors include:

- **Public Health Impact**: Very High. The single most important source for national and sub-national health system performance monitoring and resource allocation.
- **Data Availability & Maturity**: Sources with good quality and established extraction methods (e.g., DHIS2, eCHIS). This again allows us to start working our way up towards the more immature data source systems.
- **Coverage & Frequency**: Systems with high geographic coverage and frequent updates.
- **Need for Early Use Cases**: Sources critical for the first Minimum Viable Product (MVP) use cases defined in Phase 2 of the implementation roadmap.

.

## Source System Discovery & Assessment
## Template Example Use Case

**Source Name:**

Dagu Inventory System

### Phase One: Categorize by eHA Function

An inventory and logistics management information system that manages store-based transactions (requisition and issue fulfilment) and allows facilities to generate commodity requests to hubs.

**eHA Category:** Point of Service System

### Phase Two: Profile for Key Attributes

| Profiling Attribute | Attribute Description |
|---|---|
| System Name | Dagu Inventory System |
| Description | Tracks, manages, and reports on health commodity inventories across the supply chain from central warehouses to regional and facility level stores. Provides end-to-end visibility of stock levels, expiries, and movements to prevent stockouts and overstocking. |
| Source | Internal (Owned and operated by Ethiopia MoH) |
| Type | Transactional and aggregate logistics data (e.g., stock receipts, issues, balances, expiries, transfers). |
| Owner | Ethiopia Ministry of Health (MoH) / Ethiopian Pharmaceutical Supply Service (EPSS) |
| Access Method | API access (REST) and SFTP for file-based extracts. |
| Data Structure | Structured. Data stored in a relational database (PostgreSQL) within the central Dagu platform. |
| Data Update Frequency | Daily to Weekly. Facilities sync data depending on reporting schedules and connectivity. Central warehouse updates are near real-time. |
| Data Volume | ~1 TB currently (estimated). Expected to increase steadily with facility expansion and integration of new commodity categories |
| Data Sensitivity | Moderately Sensitive. Contains operational and logistic data not PII or PHI. Access controls required to protect stock and procurement data integrity. |
| Historical Data Load Plan | One-time bulk export from the Dagu Hubs (via API or SFTP) of all inventory |

| | |
|---|---|
| | transactions, facility stock balances, and master data (items, facilities) |
| Incremental Update | Daily batch ingestion using (load date or equivalent) fields to capture only new and updated records. |
| Reliability | High. Centralized system with good uptime and data consistency. Facility-level reporting may vary with internet connectivity but syncs are reliable once connected. |
| Constraints | Connectivity issues at remote facilities can delay updates. |
| Source Data Retention Policy | Long-term retention (10–15 years) for auditability, supply chain performance tracking, and trend analysis of consumption and stock movements. |
| | |

### Phase Three: Prioritize by Strategic Value

**Public Health Impact:**
Core system for pharmaceutical and commodity logistics at the facility level, enabling stock visibility, automated requisitions, and integration with dispensing systems to support continuity of care and reduce stockouts.

**Data Availability & Maturity:**
High. Implemented in over 1,294 facilities, with structured transactional data, interoperability with eAPTS, and established electronic reporting (eRRF) that enables reliable, consistent data extraction.

**Coverage & Frequency:**
High. Covers a significant portion of health facilities with daily, near real-time updates on inventory levels, commodity consumption, and dispensing activities.

**Need for Early Use Cases:**
Critical. Essential for MVP use cases in commodity management (e.g., tracking stockouts, forecasting demand) and linking supply chain data with service delivery indicators (e.g., ART dispensing, malaria test availability). Its individual level pharmaceutical transaction data supports patient level continuity of care analytics.

## Designing the Data Lake

The data lake or staging layer will be the critical first point of structured longevity within the Ethiopian National Health Data Warehouse (NHDW) architecture. This layer will have to embody the project's core principle of establishing a single, auditable source of truth by capturing a replica of every ingested dataset before any transformation, cleaning, or business logic is applied. Its design will be shaped by the High Level Requirements Document, which mandates a scalable, secure, and FAIR (Findable, Accessible, Interoperable, Reusable) data foundation. The architecture, as outlined in Chapter 6, positions the data lake as the central repository for raw, unstructured, and structured data, acting as the indispensable staging area for the subsequent data warehouse & data mart layers.

After a clear understanding of the source and systems and how we expect them to act. We'll use a framework for designing the data lake's staging area, ensuring it is purpose built to handle the complexity identified during source data profiling. The approach is built through three key steps, using the design principles from the high level requirement document.

The first step is to define the core principles and requirements that will govern the staging area. Before any technologies are chosen or folder structures are designed, it is crucial to restate the foundational purpose by drawing from the source analysis and the NHDW requirements (specifically Ch. 6.5.1, 10.3) These principles as outlined by the document are:

### I. Preserve Raw Fidelity

The preservation of raw fidelity, means data must land in its native, unaltered format, whether it be JSON, CSV, or database dumps. No business rules or transformations are applied at this initial stage, which aligns with Data Vault 2.0's principle of separating raw data ingestion from business rule application. This "as-is" approach is justified by the immense diversity of the sources; for instance, altering data from eCHIS's CommCare JSON or DHIS2's relational aggregates would destroy valuable context and make future debugging nearly impossible.

### II. Highly Restricted Access

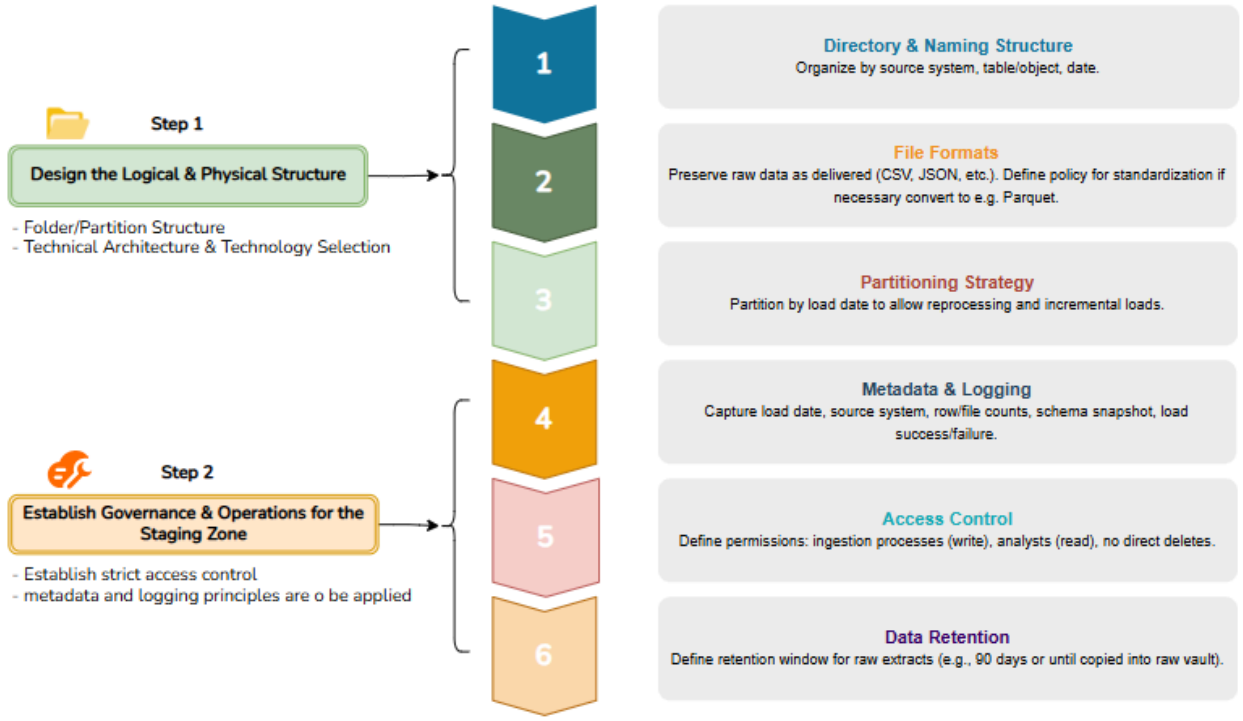
We'll treat the landing zone as a high-security zone. Where access would be strictly limited (e.g. only data engineering or authorized personnel will receive access). All data, especially PII/PHI flagged in the source profiles will be encrypted both at rest and in transit. This is a step as clearly stated by the NHDW cod (Ch. 10.3) to minimize security exposure. This will likely be implemented for more than 70% of the data sources as they will likely contain highly sensitive information.

### III. Comprehensive Discovery & Profiling

The primary purpose of preserving raw fidelity is centralization for discovery. The structure must facilitate this. Metadata (technical and business) must be captured automatically upon ingestion. This is directly specified in the NHDW roadmap **Phase 1 goal**: "***Enable detailed data discovery of all data sources in the central place*"**.

### IV. Scalability & Cost-Effectiveness

The design data lake will be built to accommodate massive, heterogeneous data volumes such as Dagu's or eCHIS's 2TB+ footprint. Raw data should be stored in cost-effective storage solutions like MinIO or HDFS, with lifecycle policies in place to archive older data once it has been processed, ensuring storage remains optimized over time. This approach is justified by source profiling that includes both current volume estimates and growth projections, such as the expectation that eCHIS data will double by 2030.



## Designing the Logical & Physical Structure

### I. Directory and Naming Structure

The physical and logical organization of the staging layer is extremely critical. Data will be organized in a predictable, manageable, hierarchical object prefix structure that immediately conveys both its origin and context at a glance. Each data object key (prefix) will follow a standardized path format, allowing engineers to easily locate, interpret, and audit data without relying on external metadata alone.

The structure will reflect project domain, source system, and data entity, with temporal partitioning to indicate ingestion or event date. This the prefix-based organization achieves the same functional outcome and integrates cleanly with external cataloging layers (e.g., Hive Metastore)

### II. Partitioning Strategy

Data partitioning will be logical, implemented through object prefix naming rather than physical directories. The pattern will organize data first by project or domain, then by source system, then by data entity (e.g., table or topic), and finally by temporal partitioning using ingestion or event dates.

**For example:**

/point_service_systems/echis/patient_visits/event_date=2022-01-01/data/part-0001.parquet

This layout supports incremental loading and easy re-ingestion or reprocessing of historical data when business rules or source corrections occur. MinIO's object immutability model will be leveraged with versioning enabled rather than overwriting, corrected or updated datasets will be re-uploaded under new versioned prefixes, preserving full lineage.

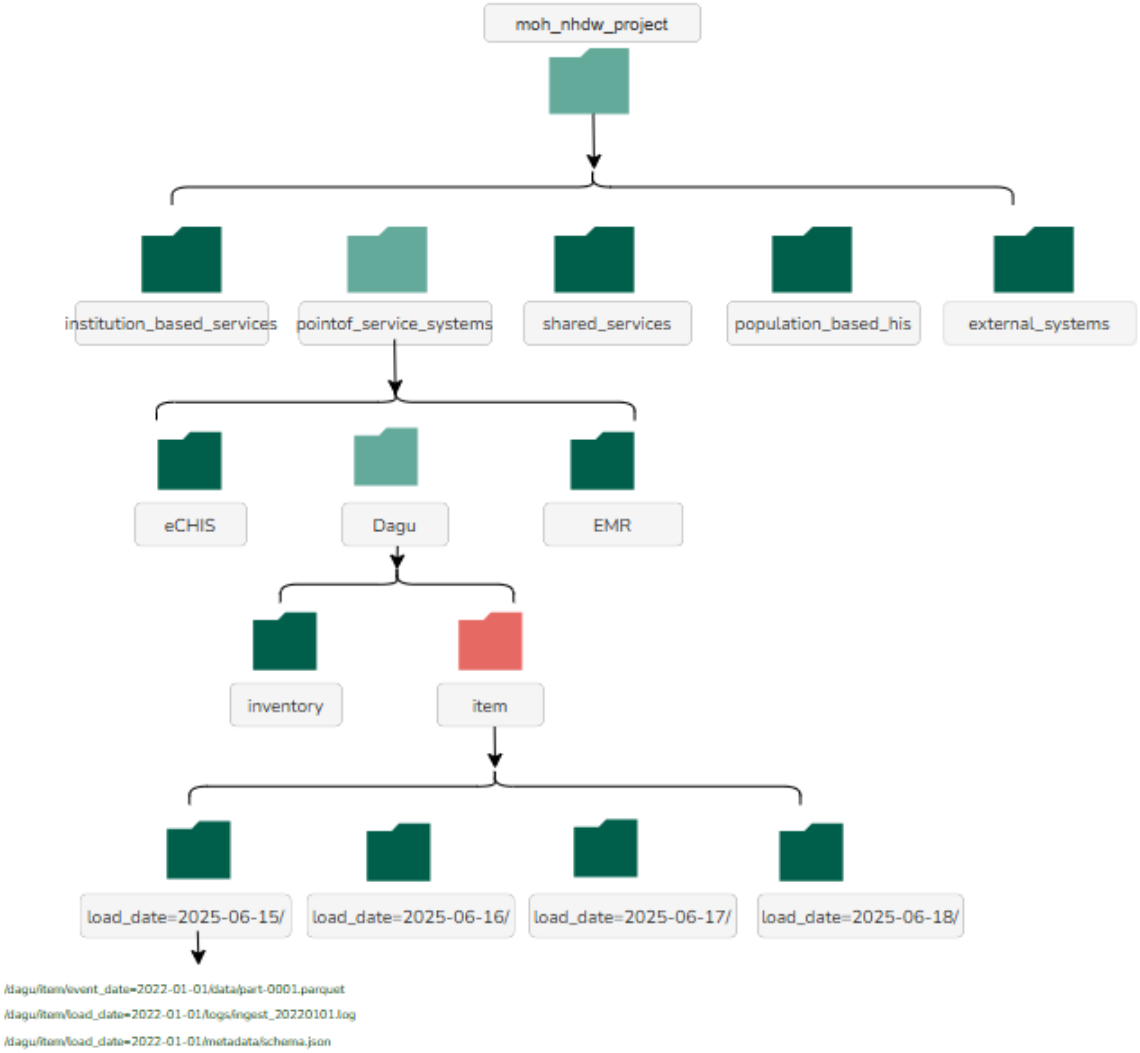
Naming conventions will be standardized and strictly enforced: lowercase letters, underscores for word separation, and no spaces or special characters. This ensures seamless compatibility across Spark, and Airflow ingestion pipelines, while reducing downstream ingestion and cataloging errors.

### III. File Formats

The file format policy will be equally important as the above. The rule will be to preserve the data in the exact format as it was received from the source be it a CSV dump from a legacy system, a JSON stream from an API, or an Avro file from a modern application.

Raw data is initially stored in its native format. For high-volume sources (>1 TB/year), a post landing optimization step will convert data to columnar format (e.g., Parquet) for performance, while retaining the original in a separate /raw/native/ subfolder for auditability.

*An example object path structure for the data lake:*



## Governance & Policy for the Staging Layer

The governance of the staging layer is its most critical aspect, directly addressing the stringent security and privacy requirements (FR7, NFR4) mandated by Ethiopian law, including the Personal Data Protection Proclamation No. 1321/2024. This layer, by its nature, will contain the highest concentration of raw, potentially sensitive Personally Identifiable Information (PII) and Personal Health Information (PHI).

### I. Metadata & Logging

All the data landing activities will be accompanied by a comprehensive metadata capture. For every object(file) or batch ingested, the system will automatically log the source system name, the exact time of the load, the number of records or files processed, a snapshot of the schema (where applicable), and the success or failure status of the operation/ingestion as demonstrated in the above image. This metadata will act as the backbone of data lineage, allowing any piece of data in the final data marts to be tracked back to its origin in the staging layer. This metadata will be stored alongside the data in a companion metadata object.

### II. Access Control

The data lake, by its nature, will contain the highest concentration of raw, potentially sensitive and Personally Identifiable Information (PII). Therefore, access must be ruthlessly controlled. A principle of least privilege is non-negotiable. Only the automated ingestion pipelines and a highly restricted team of authorized data professionals will have read and write access.

Business analysts and data scientists must be barred from this zone to prevent accidental corruption or unauthorized access to raw, unmasked data. In addition to strict access protocols, we'll be implementing robust bucket policies, IAM users/groups or policy based access control to enhance security within the data lake.

### III. Data Retention

The data retention policy for the data lake (staging layer) will be to ensure data will be stored **indefinitely**, serving as the foundational layer of data lineage within our data warehouse system.

## Bulk Data Extraction & Ingestion Phase

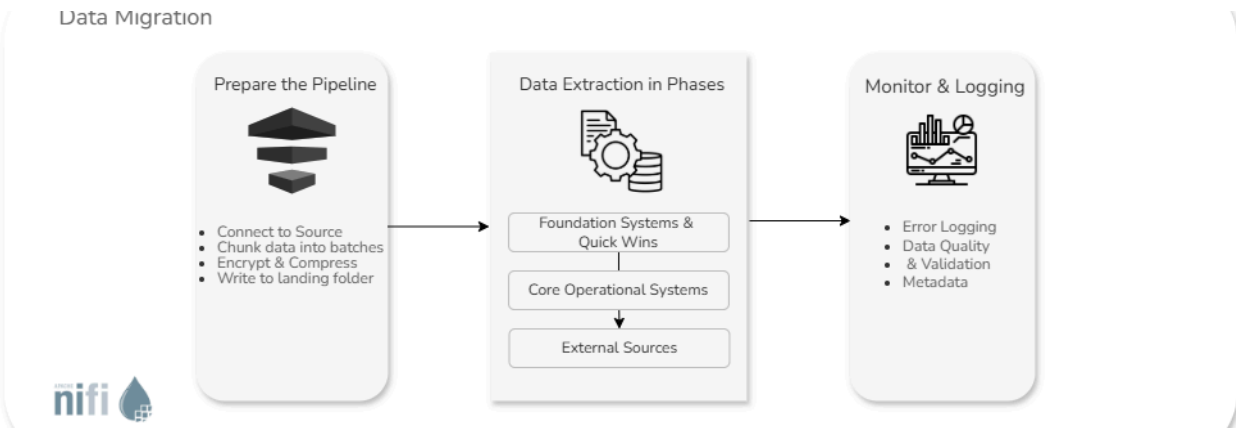
The data ingestion stage is divided into two categories. First, we will migrate a large portion of the data from the source systems into our system. Next, we will build pipelines for the scheduled ingestion of data from each source system.

After having a clear map of what data sources exits, how they behave, and where they should land in our secure staging zone, the next step is to move the historical or "full" datasets from these sources into the data lake, which is exactly what we''ll do at this stage of the process. This is not about small, daily updates, this is about **copying** the entire existing historical record from the prioritized systems in one or a few large scale operations. This will be done before we start incremental ingesting data from these source systems.

This phase is explicitly called out in the NHDW High-Level Requirements (Section 10.3: Phase 1: Prioritise centralization of raw data) with the goal: "*to have at least one complete copy of every data source that is eligible for the NHDW.*"

We propose using a visual, enterprise-grade data orchestration tool such as **Apache NiFi/Airbyte**. It has a visual interface that allows technical and non-technical team members to understand the process, while its robust & gracefully handles the realities of complex needs, including network failures and errors. It offers enterprise-grade security & full auditability of every transaction.

To manage the complexity of this national-scale data ingestion, we employ a proven three step framework. This repeatable process will be employed from the ground up to minimize risk, maximize transparency, and ensure a successful outcome for all involved, technical and non-technical alike. We'll start by preparing the pipeline, extract data in phases, and monitor and log the process.



## I. Prepare and Build the Data PIpelines

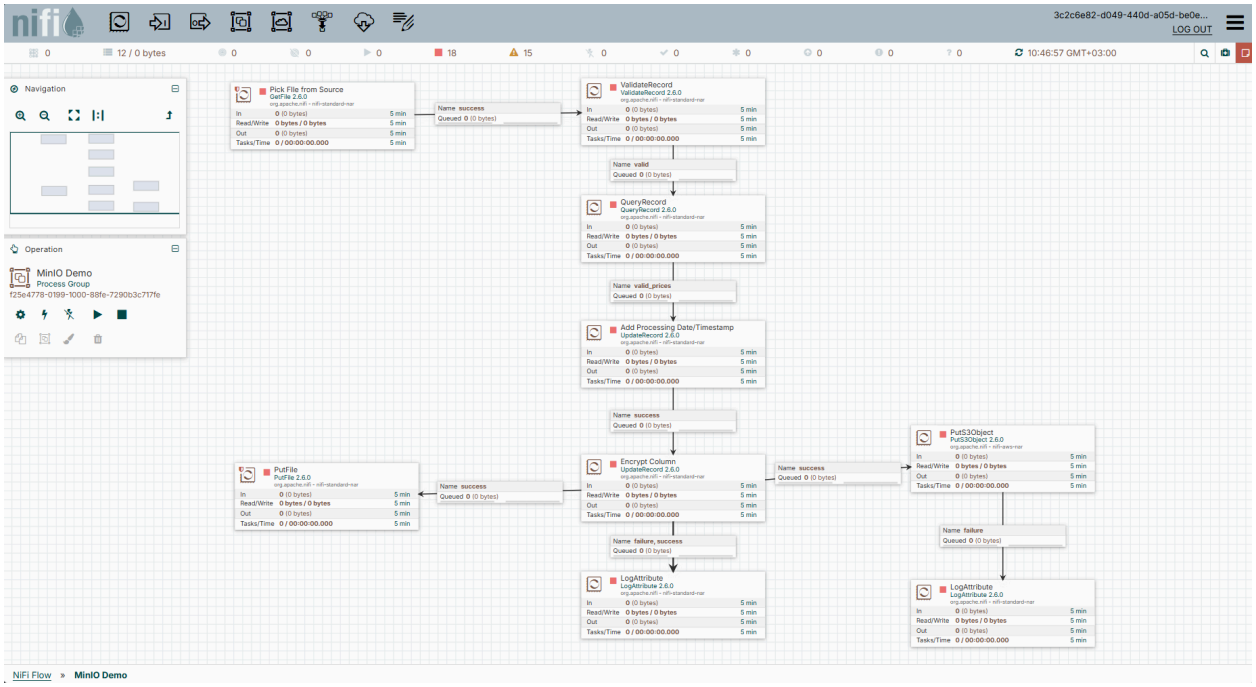
Guided by the detailed source profiles we've created, we configure dedicated dataflows for each source system (e.g., eCHIS, DHIS2, SmartCare). This configuration will directly address key questions: where data originates, its format and size, its sensitivity level, and how we will validate its arrival.

**For instance, the eCHIS Bulk Ingestion Dataflow will be configured to:**

- Connect securely to the **CommCare Sync API**.
- Extract data in small, manageable batches to respect API limits and prevent timeouts.
- Encrypt and compress each batch to ensure security where applicable.
- Deliver the data to its predefined, organized folder in the data lake (e.g., /echis/point_service_systems/patient_visits/event_date=2025-06-01/).
- Log the outcome of each batch and
- Apply data validation such as record counts, checksums, and sample field-level reconciliation e.g. date ranges, code distributions.

This entire process will be presented as a simple with tools such as **Airbyte** or **Apache NiFi**.

For example in Apache NiFi, these steps will be handled with what's called Apache NiFi process boxes. where green indicates success and red flags errors, providing an immediate, at-a-glance status update for anyone monitoring the operation.



## II. Execute a Phased and Validated Ingestion

Moving all of the data all at once in the duplication phase of the project is highly inadvisable and could be chaotic or even risky. Instead, we extract the data in phases. Carefully planned sequences based on our earlier prioritization of source systems.

### Phase One: Foundational Systems & Quick Wins

DHIS2, Dagu and MFR will be prioritized due to their role as reference data sources that enable entity resolution for downstream systems (e.g., linking eCHIS visits to facilities). These datasets are well-structured, relatively small, and provide immediate value by establishing a foundation for linking data from other systems. This stage will help in establishing stable and scalable pipelines and can potentially provide immediate value for early analytics work.

### Phase Two: Core Operational Systems

Next, we focus on the largest and most valuable datasets, such as the 2TB+ eCHIS and the critical SmartCare HIV patient records. These are essential for the MVP use cases. For this phase, validation goes beyond simple record counts to include sampling data to ensure its integrity.

### Phase Three: External Source Systems

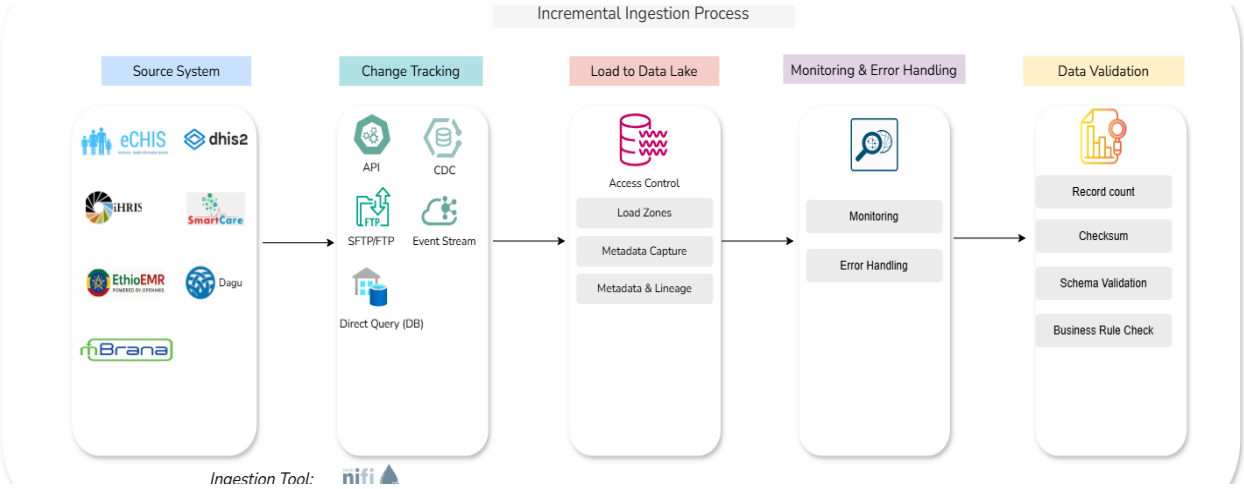
Finally, we integrate external data from partners like the Ethiopian Statistical Service or meteorology agencies. These sources often require unique handling and are best tackled once the core Ministry of Health systems are stable.

## III. Monitoring and Logging

A simple, Monitoring Dashboard offers a real-time view of progress for all stakeholders, displaying completion percentages, transfer speeds, and any errors. If an issue like a network glitch interrupts a transfer, the ingestion pipeline is configured to automatically retry and will only alert the operations team if human intervention is needed. Every action will be captured in automated logs, creating a detailed audit trail of what was moved, when, and by what system.

## Incremental or Periodic Data Ingestion Plan

Once the foundational historical datasets have been securely landed in the staging layer, shift to establishing a sustainable, reliable, and intelligent mechanism for keeping the National Health Data Warehouse (NHDW) current. This phase is a continuous, automated process that captures new and adopted records from source systems as they become available, ensuring the data warehouse reflects the most up-to-date information about the health system.




This phase is not a one-time activity but an operational element built into the core of the NHDW. As emphasised in the High-Level Requirements (Section 3.2.2), over 60% of Ethiopia's health data sources provide real-time or near-real-time updates. The architecture, therefore, will be flexible to handle everything from daily DHIS2 aggregate reports to a real-time feed from lab results.

Our approach is guided by the detailed source profiles created in phase one of the methodology. Each source system has unique characteristics, including its update frequency, data structure, and technical capabilities. All of these dictate the most appropriate ingestion pattern for each source. We will not force a single solution onto all sources; instead, we apply a fit-for-purpose strategy that aligns with the principles of the Data Vault 2.0 methodology, which prioritises flexibility and auditability.

To achieve this, we structure our incremental ingestion around three key principles:

### I. Pattern Selection Based on Source Capability

Not every system can or will have the ability to push data in real time. Thus, our source profiling template has already identified the optimal ingestion pattern for each source system. We will implement patterns as needed.

- **Change Data Capture (CDC)**: Capture only the changes (inserts, updates, deletes) since the last ingestion. This makes it highly efficient and low-impact on the source system and aligns with the real-time data ingestion (Section 6.4.2) as per the requirement.
- **Scheduled API Polling**: For systems like eCHIS (via CommCare Sync API) which expose RESTful endpoints, we will configure secure, authenticated API calls on a defined schedule (e.g., every 6 hours).
- **File Based Drop**: We will establish secure SFTP or file-sync drop zones
- **Database Query Based Extraction:** For databases without CDC capabilities, older database systems that don't support CDC but contain critical health data (e.g., older EMR systems, lab information systems), we'll implement scheduled query-based extraction.
- **Event Based Ingestion:** For systems that generate events we'll implement queue-based ingestion.

This multi-pattern approach ensures we respect the technical realities of each source while still meeting the NHDW's need for timely, accurate data.

### II. Automated Pipelines

All incremental ingestion will be orchestrated through Apache NiFi, the same robust, visual dataflow tool used in the bulk extraction phase. This provides continuity, reusability, and a unified operational view.

Each incremental pipeline will be designed with:

- Automatic retries for transient failures (e.g., network glitches).
- Backpressure handling to prevent overwhelming the system during data spikes.

Every pipeline will be fully observable through a centralized monitoring dashboard that shows real-time data flow status (running, stalled, or failed), throughput, latency from source to NHDW, and error rates. This isn't just for engineers; it also provides the Ministry of Health and its stakeholders with transparency, enabling performance monitoring and proactive issue resolution as outlined in Section 6.8.

### III. Source System Driven Scheduling

The ingestion schedule for each source will use the output of the Source System Profiling Template.

**For example:** A system profiled as “Near Real-Time” with “High Public Health Impact” (like eCHIS) will be scheduled for frequent ingestion (e.g., every 2-6 hours). A system profiled as “Monthly Aggregate” (like some DHIS2 datasets) will be scheduled accordingly.

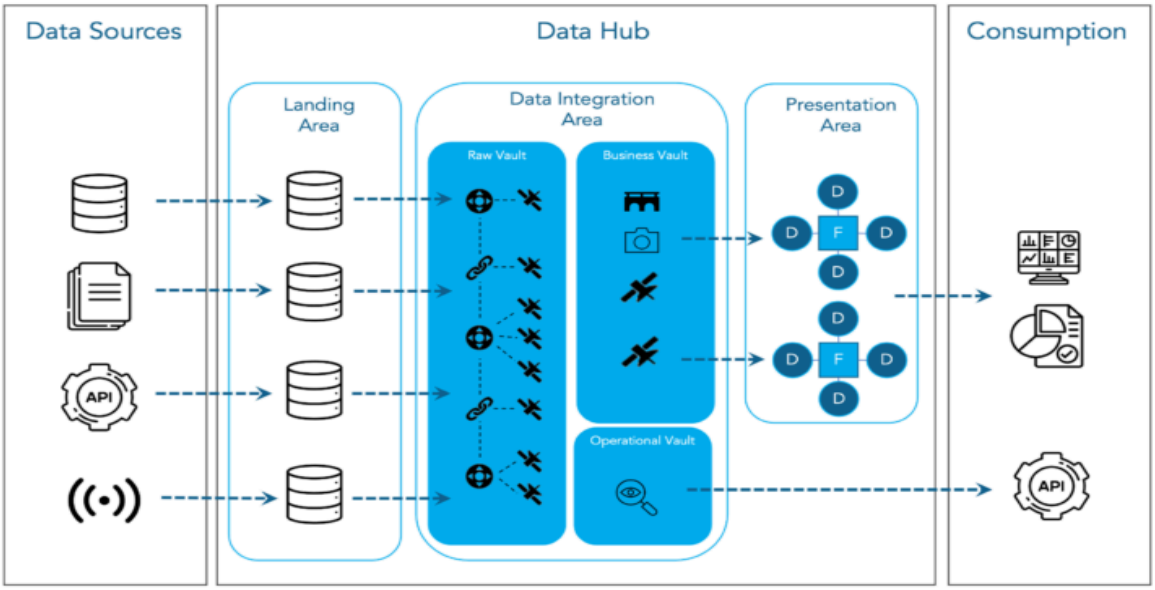
This approach ensures that our operational decisions are traceable back to strategic priorities defined by the MoH Technical Working Group. Furthermore, all incremental loads will be treated with the same rigor as the initial bulk load:

- Data is landed in its raw form in the staging layer, preserving fidelity.
- Comprehensive metadata is logged for every batch, including source system, load timestamp, record count, and schema version.
- Access remains highly restricted, with only authorized data engineering personnel able to interact with this layer.

This disciplined process ensures that the NHDW evolves from a static repository into a living, breathing reflection of the national health system, ready to support real-time dashboards, predictive analytics, and timely public health interventions all while maintaining the highest standards of data integrity, security, and auditability as mandated by the High-Level Requirements.

## Design the Raw Data Vault

Once raw data lands securely in the staging layer/landing area, it is written into the Raw Vault which is the first layer of the Data Vault 2.0 architecture mandated in Section 2.7.4 of the NHDW High-Level Requirements. At this phase we do not apply business logic, transformations or even create analytics ready views. Instead, we structure source data into a neutral, auditable, and scalable format that preserves full history and enables traceability back to origin.




**Our Approach:**

I. **Validate & Enrich**

We start by checking data from Dagu's (inventory system's source tables) which are: *inventory.inventory, inventory.inventory_status, common.inventory_period_type, common.account, institution.institution, institution.institution_type, common.inventory_reasons, inventory.inventory_log.*

We confirm record counts, schema consistency, and completeness. Each record gets technical metadata:

- **load_date**: When the data has been ingested (e.g., CURRENT_TIMESTAMP).
- **record_source**: Where the data came from (e.g., src_stg_inventory).
- **Hash keys**: MD5 for unique IDs and change detection.

This ensures clean, traceable data ready for the Raw Vault.

### II. Map to Data Vault Entities

We map each source field to one of three components, tailored to the inventory system:

| Component | Purpose | Example |
|---|---|---|
| Hub | Tracks unique business keys. | inventory_id (from inventory), institution_id (from institution). |
| Link | Connects hubs to show relationships. | link_inventory_inventory_status (ties inventory to status), link_inventory_log_inventory (ties logs to inventory). |
| Satellite | Stores descriptive data with history. | sat_inventory (reference_number, start_date, is_active), sat_inventory_log (created_by, timestamps). |

For the inventory system, we built:

- **8 Hubs**: hub_inventory, hub_inventory_status, hub_inventory_period_type, hub_account, hub_institution, hub_institution_type, hub_inventory_reason, hub_inventory_log.
- **5 Links**: E.g., link_inventory_account, link_inventory_log_user.
- **8 Satellites**: E.g., sat_inventory (with hashdiff over all attributes like reference_number || start_date), sat_inventory_status (name, code).

This keeps the inventory model separate from the prescription system, ensuring clarity and modularity. Note that it is not mandatory to structure every source table into an equal number of Hubs, Links, & Satellites. As those are determined by business value.

### III. Load with Full Auditability

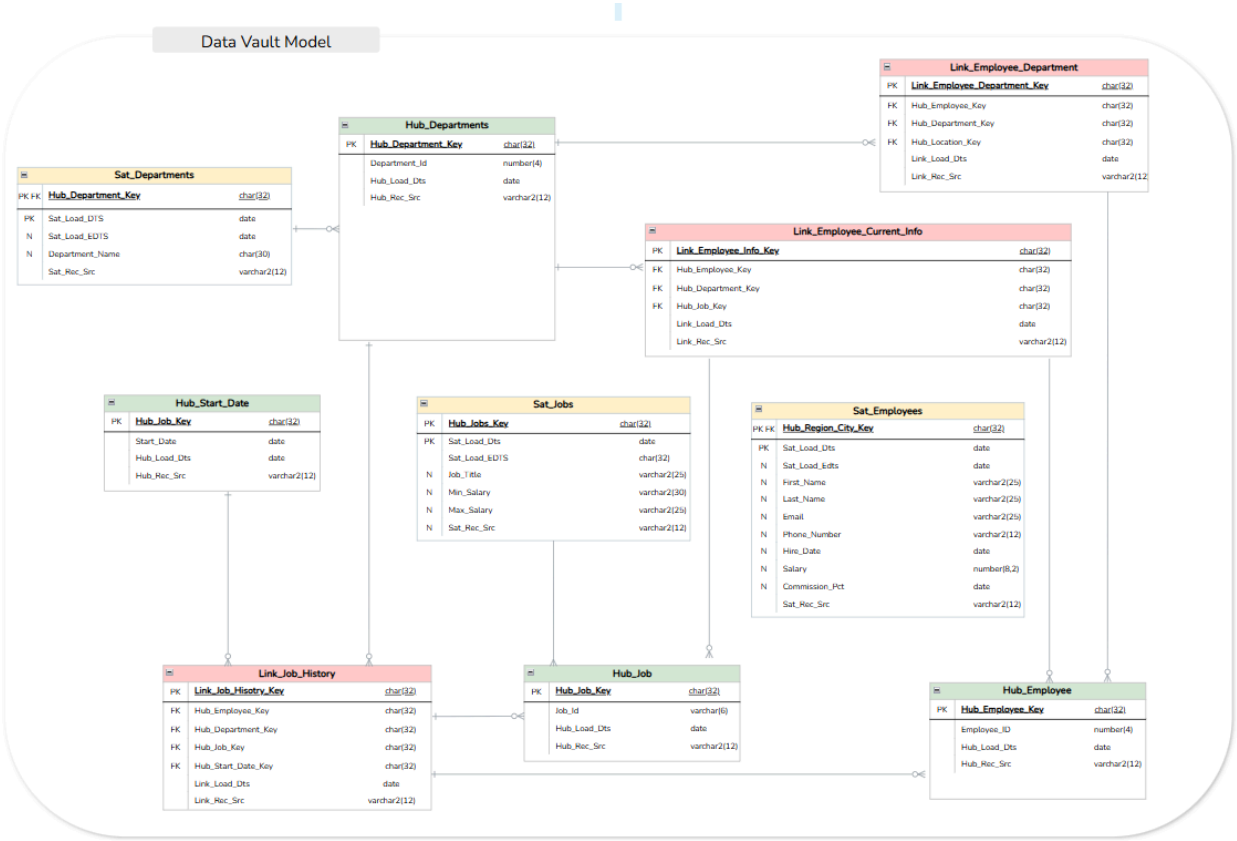
Every record will include:

- Temporal Tracking: `load_date` and `load_end_date` (for temporal tracking)
- Source metadata: `source_system` and `record_source`
- Hash keys:
    - **Hubs**: For business keys (e.g., hub_inventory_hk).
    - **Links**: For relationships between hub keys.
    - **Satellites**: Hashdiff over all descriptive fields
- Incremental loads: skips duplicates meaning, in conflict do nothing, keeping our ETL/ELT jobs fast and efficient. Staging tables will act as a buffer, transforming source data before loading into the Raw Vault.

## IV. Enforce Security & Governance

- All PII/PHI remains encrypted at rest and in transit (per Personal Data Protection Proclamation No. 1321/2024)
- Access restricted to authorized data engineers only
- Full audit logs captured for lineage and compliance

**Sample Model:**



## ELT/ETL Pipeline for Data Vaults

Once data is structured in the Raw Vault, it moves through ETL/ELT pipelines that apply business logic, enforce data quality, and prepare data for analytics. These pipelines are the engine that turns raw records into trusted, analysis-ready information.

As per section 6.4 of the NHDW requirements, ELT must support batch, near-real-time, and event-driven ingestion, with strong auditability, error handling, and metadata tracking. This allows our approach to be standardized.

All ELT pipelines follow a repeatable sequence:

- **Read from Raw Vault**: Pull hub, link, and satellite records for the relevant source. e.g., Dagu inventory transactions.
- **Apply Business Rules**: Apply Business Rules: Execute MoH-defined logic for supply chain KPIs. e.g., Stockout = closing_balance = 0 for ≥2 days.
- **Enrich with Master Data:** Resolve facility and product identifiers using master records from MFR & Dagu reference tables to standardize linkage across inventory domains.
- **Validate & Cleanse**: Confirm record counts, schema conformity, and completeness against Raw Vault metadata. Flag or quarantine invalid records for reprocessing.
- **Load to Business Vault:** Write results to the Business Vault as per prior model architecture using idempotent, version-controlled jobs.

| Source | Ingestion Pattern | Transformation |
|---|---|---|
| **eCHIS** | Near real-time (every 2–6 hrs) | Temporal alignment, clinical rule application |
| **DHIS2** | Daily/monthly batch | Indicator validation, denominator reconciliation |
| **External (EMI)** | Daily batch | Geospatial join, temporal lag alignment |

**Governance & Reliability**

- **Metadata tracking**: Metadata tracking: Each pipeline logs source_system, record_source, load_date, rule_version, and record counts, maintaining full lineage from Raw to Business Vault.
- **Failure handling**: Automatic retries, backpressure control, and alerts for manual intervention.
- **Security**: PII/PHI remains encrypted; only de-identified or aggregated outputs leave the secure zone.
- **Tooling**: Built using open-source, metadata-driven ELT frameworks (e.g., Apache Spark & Airflow), ensuring flexibility and avoiding vendor lock-in.

## Business Vault Design

The Business Vault is where MoH-defined business logic is applied to Raw Vault data to create trusted, analysis-ready entities. While the Raw Vault captures source data as-is, the Business Vault applies standardized MoH logic to create consistent, auditable entities ready for analytics.

This layer will directly support the hybrid architecture and Data Vault 2.0 (Section 2.7.4) principles, enabling agile, auditable, and scalable analytics.

### Core Design Principles

- **Separation of concerns:** Business rules are applied after raw data is stored, allowing rules to evolve without re-ingesting source data.
- **Versioned logic:** Every business rule (e.g. case definitions) is documented, versioned, and traceable.
- **Reusability:** Common logic (e.g., "facility catchment area") is shared across use cases.

### Modelling Approach

We extend the Raw Vault structure with Business Hubs, Links, & Satellites:

| Component | Purpose | Example |
|---|---|---|
| Business Hub | Canonical entity with resolved identifiers | Inventory hub linked via MFR & Dagu master data across warehouse, facility, and transaction systems. |
| Business Link | Validated relationships based on business rules | Inventory_Link = (Facility + Product + Period) where closing_balance = 0 and reorder_status = pending. |
| Business Satellite | Attributes derived from business logic | Stock_Movement_Status = 'Delayed' if delivery_date - request_date > threshold_days. |

## Designing the AI & Consumption Layer

This layer will be designed to empower data use-cases for various stakeholders. It will operationalize the NHDW's dual mandate: enabling **evidence-based program management** and powering **predictive analytics**. The ultimate goal being to make sure data scientists, data analysts, medical researchers and others can leverage the extensive data within the Data Lake for advanced analytical and AI driven applications. It will also be designed to serve business users, including managers, monitoring and evaluation (M&E) specialists, directors, and executives.

This layer will serve as a platform to two interlocking components:

1. **Data Science & AI Platforms:** this facilitates data mining, machine learning, predictive modelling, and other data science activities. This means raw data is now transformed into actionable insights that drive health care innovations and preventive care. Using the comprehensive data from the storage layer (data lake or data warehouse) for versatile use-cases and models for various applications, including disease prediction, early treatment, and personalised treatment.
2. **Data Presentation Platform:** provides visualising and analysing capabilities to the platform driving evidence based decision making and effective policy-making. All insights whether from a simple dashboard or a predictive model are delivered through secure, role-based interfaces. User friendly dashboards and reports with real time capabilities to enable decision makers prompt responses to emergy trends and issues.

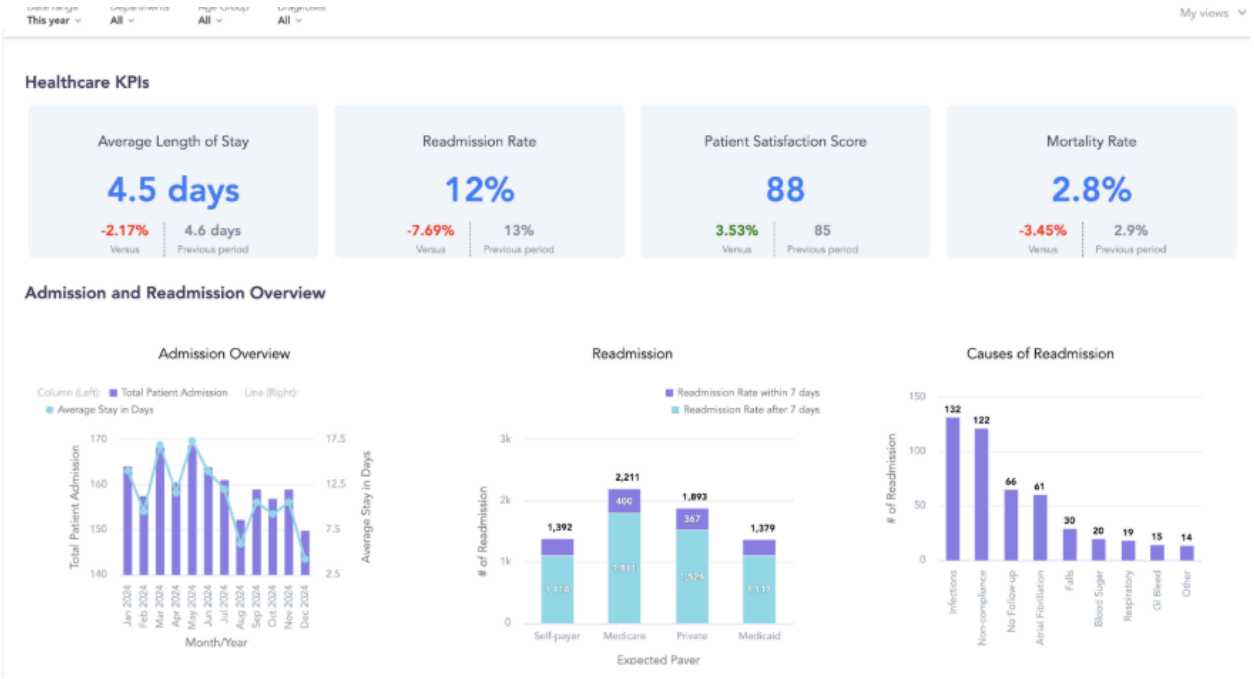
Ultimately we'll treat this layer as the end product the stakeholders get to consume. This is where we take the NHDW from a run of the mill standard data warehouse into a transformative asset to the MOH that is enhanced with both realtime and AI capabilities. Showcasing its true intent and purpose from vision to reality. Together with

In order to empower this layer, purpose-built data marts, views, and virtual tables will facilitate subject-specific datasets that power dashboards, reports, realtime and advanced analytics for MoH programs. Per Section 6.6 data marts will support interactive exploration, real-time visualization, longitudinal analysis, and should have secure API access.

We build data marts using a **dimensional modelling** (star schema) optimized for fast querying and intuitive use:

| Component | Purpose | Example |
|---|---|---|
| Fact Table | Measures and metrics | Malaria_Cases :count, test positivity rate, deaths |
| Dimension Tables | Context for analysis | Facility (MFR), Patient_Info(age, sec, location) |

## Example Dashboard




### Key Features

- Use Case: Each mart maps to a prioritized MoH use case
- Performance: partitioning by time/facility enables sub-second queries.
- Security:
    - PII/PHI is masked or aggregated per Ethiopian Data Protection Proclamation
    - RBAC ensures analysts only see data for their program/region.
- Access Methods:
    - Dashboards (eg. Microsoft Power BI, Apache Superset, Google Looker) for program managers
    - SQL interfaces for data scientists

## Designing the Master Data, Metadata & Governance Layer

This layer ensures the consistency, accuracy, and accountability of core data entities across the healthcare ecosystem. The implementation of MDM facilitates the effective management and harmonisation of key data entities such as clients, healthcare facilities, and healthcare workers. The purpose of this crucial layer is to ensure interoperability and operational efficiency with not only the National Health Data Warehouse but also within the health sector.

This layer is foundational to the integrity, usability, and trustworthiness of the entire National Health Data Warehouse (NHDW). As mandated in Chapter 6.7 of the High-Level Requirements, it ensures consistency, accuracy, and accountability of core health data entities across disparate source systems. Without this layer, data integration would be fragmented, analytics unreliable, and cross-system interoperability nearly impossible.

The Master Data, Metadata & Governance Layer is the proactive enabler of the FAIR principles (Findable, Accessible, Interoperable, Reusable) and a prerequisite for high-quality analytics, regulatory compliance, and ethical data use. Its design directly supports the hybrid architecture and Data Vault 2.0 methodology by decoupling identity resolution and semantic standardization from business logic.

### Purpose and Strategic Alignment

This layer serves three interlocking purposes:

#### Master Data Management (MDM)

Establish and maintain a single, authoritative version ("golden copy") of core entities such as patients, facilities, health workers, and products. This enables accurate entity resolution across systems (e.g., linking an eCHIS household visit to a facility in DHIS2 via a standardized facility ID from the MFR). This allows us to hold a verified source of truth for every analysis and model to compare and validate on.

#### Metadata Management

Capture, store, and expose technical, operational, and business metadata including data lineage, schema definitions, ownership, and usage to support data discovery, impact analysis, and auditability.

#### Data Governance

Enforce policies for data quality, privacy (per Proclamation No. 1321/2024), access control, and stewardship across the NHDW lifecycle.

This aligns directly with Phase 3 of the Implementation Roadmap (Section 10.5): "*Consolidate*

*data discovery... establish foundational master data... enable high-quality data standards and mapping.*"

## Core Components

We will implement the following registry-based services, building on existing national assets wherever possible:

### Master Facility Registry (MFR)

Serves as the single source of truth for all health facilities capturing name, location, service profile, and operational status. Already mature (~80% populated), it will be integrated as the authoritative facility reference for all NHDW pipelines.

### Client Registry / Master Patient Index (MPI)

Uniquely identifies individuals across systems (e.g., eCHIS, SmartCare, LIS) to enable longitudinal patient analytics. This registry will leverage the National ID (Fayda) as a common identifier where consent and policy allow, or use probabilistic matching with privacy safeguards where not.

### Health Worker Registry (iHRIS)

Centralizes credentials, roles, and facility assignments of health professionals critical for workforce analytics and service attribution.

### Terminology Service (e.g., NHDD, OpenConceptLab)

Harmonizes clinical and operational codes (e.g., ICD, SNOMED CT, local HMIS indicators) to ensure semantic interoperability. This service will map source-specific codes to a canonical reference terminology.

### Data Catalog & Lineage System

A centralized inventory of all data assets in the NHDW, powered by tools like Apache Atlas or DataHub. It will auto-ingest metadata from the staging layer and pipelines, enabling users to discover datasets, understand their origin, and assess fitness for use.

### Implementation Approach

Our strategy follows a "build on existing, govern centrally" principle:

- **Assess & Integrate**: Begin with an inventory of current registries (MFR, iHRIS, Fayda, NHDD). Where they exist and meet quality thresholds, we integrate them as authoritative sources not replace them.
- **Reconcile & Resolve**: For entities lacking a national registry (e.g., patients), implement data reconciliation workflows in the Raw Vault to build a trusted MPI. Deduplication logic will be versioned and auditable.
- **Enforce at Ingestion**: During ETL, all pipelines will resolve business keys (e.g., facility_code) against the MFR before loading into the Business Vault. Failed resolutions will be quarantined for stewardship review.
- **Automate Metadata Capture:** Every batch ingestion will auto-populate the data catalog with source, schema, record counts, and lineage ensuring the NHDW is self-describing from the start.
- **Assign Data Stewards:** Per Chapter 8, MoH will appoint domain-specific stewards (e.g., from EPSS for commodities, EPHI for labs) to own data quality rules, business definitions, and approval workflows for their subject areas.

### Enabling Trust and Compliance

This layer operationalizes Ethiopia's Health Data Governance Principles and the Personal Data Protection Proclamation No. 1321/2024:

- **Data Quality by Design:** Automated profiling, validation rules, facility ID must exist in MFR, and anomaly detection will be embedded in pipelines.
- **Auditability**: Full lineage from source system → staging → Raw Vault → Business Vault → data mart will be captured and exposed via the catalog.
- **Privacy by Default**: PII/PHI linkage to National ID will be stored separately from clinical data and only joined under strict RBAC and audit logging.
- **Stewardship Workflow**: Data stewards will use a governance console to review data quality alerts, approve master data changes, and document business rules such as, What is a '*stockout*'?

## Designing the Monitoring and Management Layer

This layer will be the central system for operational health, performance, and accountability. It provides realtime visibility into every component of the NHDW from data ingestion pipelines to query performance ensuring reliability, observability, and trustworthiness. In a system as complex and mission-critical as Ethiopia's NHDW ingesting data from more than 80 source systems, supporting national health decisions, and handling sensitive patient information. Stakeholders must know it works, and engineers must see when it doesn't.

The Monitoring Layer will directly support the Non-Functional Requirement NFR6's Performance, Reliability, As it is explicitly mandated by the NHDW's architecture design, it is also critical for Phase 1's goal which is Enable detailed data discovery through monitoring. This layer is important because it detects hardware and software bottlenecks or failing nodes before they cause outages.

Our approach centers on a unified observability stack built exclusively on **Prometheus** & **Grafana**. This choice reflects a deliberate trade-off rather than a fragmented suite of monitoring tools, we consolidate all system health signals; infrastructure, pipeline, and application into a single coherent view. Prometheus collects time-series metrics from every layer of the NHDW stack; CPU and memory usage from servers, job duration and success rates from the ingestion pipeline flows, record throughput from Spark ETL processes, and query latency from the BI layer.

**For example:**
A Ministry of Health manager can view a high-level dashboard showing "% of source systems with successful daily ingestion over the last 7 days," with red/yellow/green indicators for quick status assessment. Meanwhile, a data engineer can drill into the same system to see exact flow for backpressure, JVM garbage collection pauses, or Spark stage failures all from the same underlying dataset.

Alerting is threshold-driven and tightly integrated: if the success rate of Dagu inventory ingestion drops below 95% for 15 minutes, Grafana triggers an alert to the on-call engineer via email or SMS. But this isn't just noise, it's actionable. Every alert includes a direct link to the relevant Grafana dashboard panel, enabling rapid triage without context switching.

## Core Components and Technical Implementation

We implement this layer using a combination of open-source tools for cost-effectiveness and flexibility, integrated into a unified observability platform.

### a. Infrastructure & System Health Monitoring:

This layer continuously scrapes metrics from all servers, containers, and services (e.g., CPU, memory, disk I/O, network bandwidth, service uptime). Using Prometheus, pulls metrics at defined intervals. Grafana consumes this data to build dynamic dashboards.

Key Uses:

- **System Load:** CPU Utilization, Memory Usage, Disk Space (free/used), Network Latency.
- **Service Health**: Uptime of Spark, Airflow, and database services.
- **Data Flow:** Throughput (GB/s) for ingestion and ETL jobs.

### b. Application & Pipeline Performance Monitoring:

This process tracks the performance of our core data processes. We instrument our ingestion flows and jobs to emit custom metrics, enabling us to monitor ingestion success rate in the percentage of batches ingested without error.

Key Uses:

- **Ingestion Success Rate**: number or percentage of batches ingested without error
- **ETL Job Duration:** Time taken for each pipeline run successfully
- **Query Latency:** Average time to return results from the Data Mart layer. This helps identify slow or failing data pipelines, allowing engineers to optimize or fix them.

### c. Log Aggregation & Analysis (Grafana Loki)

This stage centralizes all logs from Airbyte/NiFi, Spark, databases, and applications into a single searchable repository. It works by configuring all components of the architecture to send logs to Grafana Optics. Ibana provides

Key Uses:

- **Troubleshooting:** Quickly find errors by searching for specific keywords or timestamps.
- **Auditing:** Track user activity (e.g., who accessed what dataset, when).
- **Compliance:** Maintain a permanent record of all system events for regulatory audits. Turning unstructured log files into structured, actionable data for debugging and compliance.

### d. Anomaly Detection & Alerting

Automatically identifies unusual patterns (anomalies) and triggers alerts with defining alert rules. e.g., “If nifi_flow_success_rate < 95% for 5 minutes, trigger an alert”). Alerts are sent via email. This is particularly useful as it moves the operation from reactive firefighting to proactive problem solving in real-time.

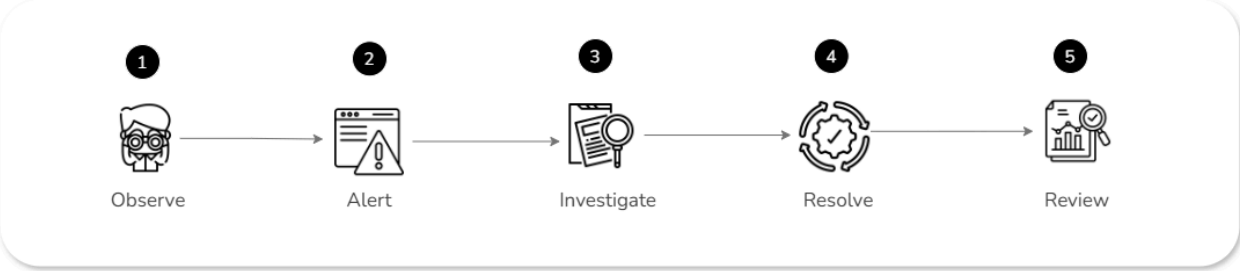
Key Uses:

- Sudden drop in data ingestion volume.
- Spike in job failure rates.
- Unusually long query times.

We build performance reporting mechanisms for automated weekly and monthly reports via Grafana dashboards. This provides stakeholders with transparent, quantitative evidence of the system’s health and efficiency.

**Operational Workflow**

The Monitoring Layer is not just about dashboards; it defines a process:




System health and data pipelines are continuously observed through centralized dashboards to ensure expected performance. When anomalies or failures are detected, alerts are automatically triggered and routed to the responsible technical teams. Engineers then investigate the issue using centralized logs and metrics, identify the root cause, and implement corrective actions. Resolved incidents are documented and reviewed regularly to identify systemic improvements and prevent recurrence.

## Designing the Security and Access Control Layer

This layer will enforce security measures and access controls to protect sensitive data and ensure compliance with regulations. Security within the NHDW is the foundation upon which trust, compliance, and utility are built. Given that over **79%** of ingested datasets are expected to contain personally identifiable information (PII) or personal health information (PHI), and that the system will serve users ranging from frontline health workers to national policy makers, a zero-trust, defence in depth approach is mandatory. This layer will ensure that only the right people, at the right time, for the right reason, can access the right data and that every access event is auditable.

### How It Works: Identity, Policy, and Enforcement

The architecture centers on centralized identity management and fine-grained access control, implemented through open-source, standards-based tooling. Rather than bottling onto applications after the fact, it is embedded into the data platform from the ground up.

#### Centralized Identity and Authentication

All users whether analysts, program managers or ETL pipelines are provisioned through a single identity provider. We deploy **Keycloak** as the core identity and access management (IAM) server, integrated with Ethiopia's existing authentication systems. e.g., the National ID/Fayda platform.

Multi-factor authentication (MFA) is enforced for all privileged roles and for any remote access. This eliminates credential sprawl and ensures consistent authentication across the entire stack from ingestion to visualization.

#### Role-Based Access Control (RBAC) with Context

Access to data is governed not just by job title, but for purpose, location & data sensitivity. RBAC policies are defined in **Apache Ranger**, which enforces access rules directly at each layer of the architecture eg., Hive, Spark, MinIO

**For example:**

- A data scientist working on ART adherence models may access de-identified longitudinal records but only after their project is approved by the Data Governance Committee and a data use agreement is in place.
- Automated pipelines: For example NiFi runs under service accounts with the minimal privileges needed to read from source systems and write to the staging zone nothing more.

These policies are dynamic: if a user's role changes or a dataset is reclassified as highly sensitive (e.g., HIV status), access is automatically revoked or restricted without manual intervention.

**Data Protection at Every State**

**In transit**: All data movement whether from a remote health post via HealthNet or from the staging layer to the Business Vault is encrypted using TLS 1.3. APIs require OAuth 2.0 tokens for every request.

- **At rest**: Sensitive fields (e.g., patient name, National ID) are encrypted using AES-256 in the staging layer. In downstream layers (Business Vault, data marts), PII/PHI is either masked (for dashboards) or removed entirely (for AI/ML sandboxes). The encryption keys are managed separately from the data, using a dedicated secrets management service.
- **In use**: Query engines (e.g.Spark SQL) are configured to enforce row- and column-level security. Even if a user bypasses the BI tool and runs a direct SQL query, they cannot access data their role doesn't permit.

**Auditability and Accountability**

Every data access event successful or failed is logged with immutable metadata: who accessed what, when, from where, and for what purpose. These logs are written to a write-once, read-many (WORM) storage system and fed into the central monitoring layer (Prometheus/Grafana). This satisfies both operational needs (e.g., "Why did this dashboard go blank?") and legal requirements under Proclamation No. 1321/2024, which mandates full audit trails for any processing of personal data.

**Secure APIs and External Sharing**

When external partners (e.g., research institutions, donors) need access to NHDW outputs, they do so through versioned, rate-limited REST APIs. These APIs never expose raw tables; instead, they serve pre-approved, aggregated views (e.g., "Malaria test positivity by zone, monthly"). Each API consumer is issued a unique key tied to a specific data use agreement, and all queries are logged and throttled to prevent bulk extraction.

Access control policies are centrally defined, versioned, and governed, while enforcement is applied natively at each data service layer. This approach ensures consistent authorization rules across the platform without introducing a single point of failure, and allows each query engine and storage service to enforce security controls at execution time in line with its native capabilities.